

GTU ROBOTICS CLUB

Docker setup:

1.Start the basic knowledge of docker

2.Setup the docker in your system :

<https://youtu.be/cqbh-RneBlk?si=VGxigQtl1LdN9a9W>

3.Set the Nvidia - smi in your host system for gpu access

See the docs of Tensorrt_cuda_deployment_pdf in GITHUB

4.Set the nvidia-container toolkit

<https://docs.ultralytics.com/guides/docker-quickstart/#prerequisites>

Go through from this website

Verify NVIDIA-SMI Command

Start the setup of NVIDIA container toolkit from this website

5. Find the best docker page

LINK: <https://hub.docker.com/r/ultralytics/ultralytics/tags>

Example : Pull this image

Cmd: docker pull ultralytics/ultralytics:latest-python-export

6. Check through command

Cmd: **sudo docker images**(for images)

Cmd: **sudo docker ps -a**(for container)

7.enter the docker images with gpu , usb, photo, video,

Start with photo

Cmd : echo \$DISPLAY

Cmd: # On host

xhost +local:root (for photo, gui access everytime run with or enter the container after above two cmd run)

8. Enter the setup of NVIDIA-SMI host system as well as then into the docker

A.cuda

B.Nvidia-smi

C.cudnn

D.Tensorrt

File from host to into the docker:

Cmd: sudo docker cp

/home/purushottam/Downloads/nv-tensorrt-local-repo-ubuntu2204-10.13.2-cuda-13.0_1.0-1_amd64.deb ultralytics_container:/ultralytics/

9. Start this basic configuration

apt-get update

apt-get install -y usbutils

Check cmd: lsusb

Start the downloading and installing library
opencv -python, pyrealsense and other necessary libraries

Install the nano for edit and make a new file

apt-get update

apt-get install -y nano

Basic nano shortcuts

- **Ctrl + O** → Save the file (writes out)
- **Enter** → Confirm filename when saving
- **Ctrl + X** → Exit nano
- **Ctrl + K** → Cut a line
- **Ctrl + U** → Paste a line
- **Ctrl + W** → Search text

Go through this website for docker as well as host system for
Realsense SDK

Link :

https://github.com/IntelRealSense/librealsense/blob/master/doc/distribution_linux.md

```

root@0694fddb00f0:/ultralytics# lsusb
Bus 002 Device 011: ID 8086:0b5c Intel(R) RealSense(TM) Depth Camera 455 Intel(R) RealSense(TM) De
Bus 002 Device 001: ID 1d6b:0003 Linux 6.8.0-83-generic xhci-hcd xHCI Host Controller
Bus 001 Device 003: ID 048d:c966 ITE Tech. Inc. ITE Device(8176)
Bus 001 Device 002: ID 5986:212b SunplusIT Inc Integrated Camera
Bus 001 Device 004: ID 8087:0026
Bus 001 Device 001: ID 1d6b:0002 Linux 6.8.0-83-generic xhci-hcd xHCI Host Controller
root@0694fddb00f0:/ultralytics# ls /dev/video*
/dev/video0 /dev/video1 /dev/video2 /dev/video3 /dev/video4 /dev/video5 /dev/video6 /dev/vid
root@0694fddb00f0:/ultralytics# ls
2025-09-23-161706.webm  README.zh-CN.md                                docker
CITATION.cff           ai.jpeg                                           docs
CONTRIBUTING.md       best.pt                                           examples
LICENSE                cuda-repo-ubuntu2204-13.0.1-580.82.07-1_amd64.deb  mkdocs.yml
README.md              cudnn-local-repo-ubuntu2204-9.13.0_1.0-1_amd64.deb  nv-tensorrt-1
root@0694fddb00f0:/ultralytics# python3 realsense_test.py
root@0694fddb00f0:/ultralytics# nano trained_model.py
root@0694fddb00f0:/ultralytics# python3 trained_model.py

0: 480x800 (no detections), 49.7ms
Speed: 3.7ms preprocess, 49.7ms inference, 16.0ms postprocess per image at shape (1, 3, 480, 800)

```

Check above all command working in inside docker container and run your first code with ultimate GPU access.(plug intel realsense as well)

Most Imp How to set TensorRT Framework in Docker

Use this code to convert it .pt file to .engine file

Cmd: python3 convert.py

```

from ultralytics import YOLO

# Load a YOLO11n PyTorch model
model = YOLO("yolo11n.pt")

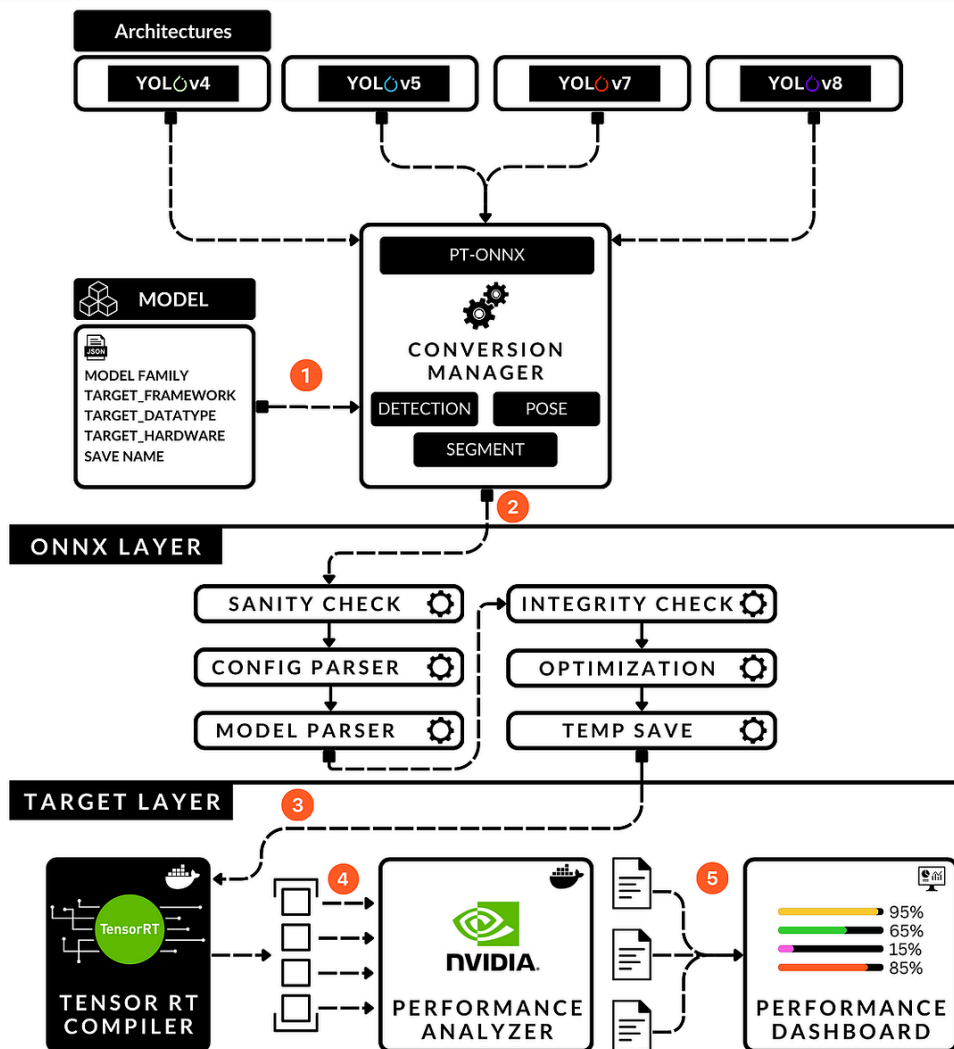
# Export the model to TensorRT
model.export(format="engine") # creates 'yolo11n.engine'

# Load the exported TensorRT model
trt_model = YOLO("yolo11n.engine")

# Run inference
results = trt_model("https://ultralytics.com/images/bus.jpg")

```

YOLO Model-Target Pipeline

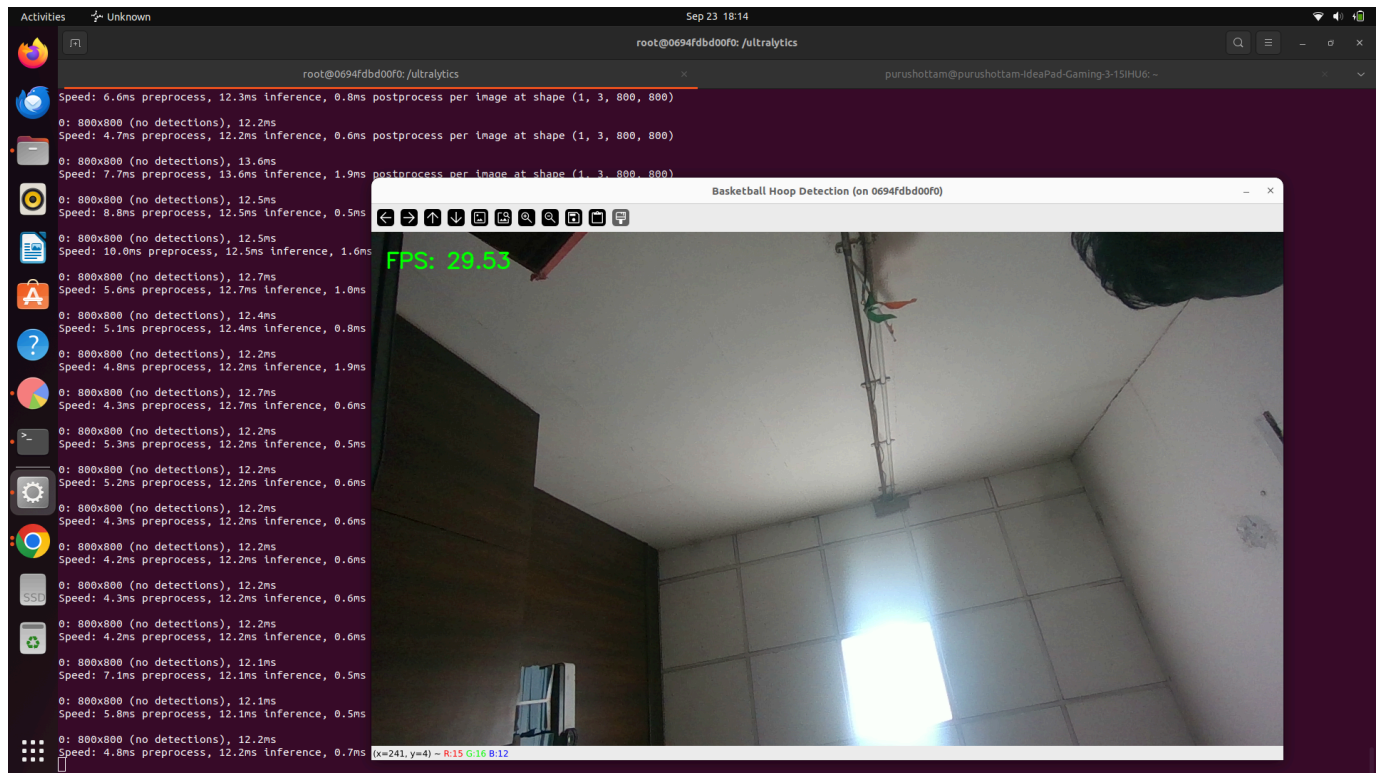


After that run you file

Cmd: python3 trained_model_with_TensorRt.py

You see something like this

This for FP32 inference



If you need FP16 inference go through this line of code
model.export(format="engine", half=True) # FP16 precision, faster
Instead of this
model.export(format="engine")